ICPC IUBAT Asia Dhaka Regional 2026 Online Preliminary Contest

Hosted By

Department of Computer Science and Engineering, IUBAT
IUBAT - International University of Business Agriculture and Technology

Supported by

You get 8 problems, 13 pages and 240 minutes.

Problem A. Whirlpool

A lake, represented by a 2D coordinate plane, has n distinct *protected spots*. The i-th protected spot is at (x_i, y_i).

Currently, there is a treasure on each protected spot. The treasure on the i-th protected spot has value w_i.

A whirlpool is about to sweep across the lake. You may place its center (c_x, c_y) at any point in the plane; the coordinates of the center do not have to be integers. The whirlpool is described by two given integers a and b. It moves a treasure at (x, y) to (x', y'), where

$$x' = c_x + a(x - c_x) - b(y - c_y),$$
$$y' = c_y + b(x - c_x) + a(y - c_y).$$

The whirlpool moves all treasures in the lake simultaneously, exactly once. A treasure is *safe* if it ends up exactly on one of the protected spots. It does not have to be the spot it started on.

Find the maximum possible total value of safe treasures over all choices of the center.

Input

The first line contains a single integer T ($1 \le T \le 20$) — the number of test cases.

The first line of each test case contains three space-separated integers n, a, and b ($1 \le n \le 1000$, $-10^4 \le a \le 10^4$, $-10^4 \le b \le 10^4$, $b \neq 0$) — the number of protected spots and the two whirlpool parameters.

The second line contains n space-separated integers $x_1, x_2, \ldots, x_n$ ($-10^8 \le x_i \le 10^8$) — the x-coordinates of the protected spots.

The third line contains n space-separated integers $y_1, y_2, \ldots, y_n$ ($-10^8 \le y_i \le 10^8$) — the y-coordinates of the protected spots.

The fourth line contains n space-separated integers $w_1, w_2, \ldots, w_n$ ($1 \le w_i \le 10^9$) — the values of the treasures.

All protected spots in a test case are distinct.

It is guaranteed that the sum of n over all test cases does not exceed 2000.

Output

For each test case, output a single integer — the maximum possible total value of safe treasures.

Sample Input	Sample Output
3 1 1 1 2 -3 7 4 0 1 0 1 1 0 0 0 1 1 1 2 3 4 3 1 1 2 2 1 1 2 3 5 4 1	7 10 9

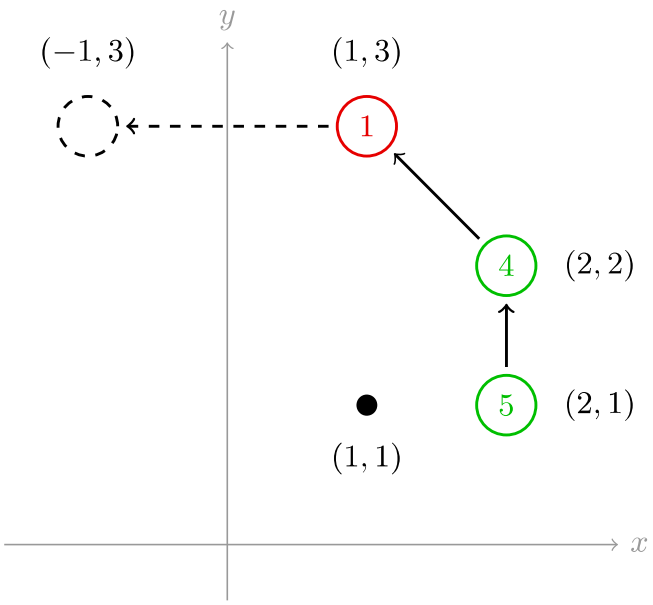
Explanation

In the first test case, place the center at the only protected spot. The treasure does not move, so it is safe and the answer is 7.

In the second test case, place the center at $\left(\frac{1}{2}, \frac{1}{2}\right)$. Every treasure moves to the next corner of the square in counterclockwise order, so all four treasures are safe and the answer is $1 + 2 + 3 + 4 = 10$.

In the third test case, place the center at $(1, 1)$. The whirlpool rotates every treasure by $45°$ counterclockwise around the center and multiplies its distance from the center by $\sqrt{2}$. The treasure at $(2, 1)$ moves to $(2, 2)$, and the treasure at $(2, 2)$ moves to $(1, 3)$, so both are safe. The treasure at $(1, 3)$ moves to $(-1, 3)$, which is not a protected spot. The total value of safe treasures is $5 + 4 = 9$. It can be proven that it is impossible to keep all three treasures safe.

The following is an illustration of the third test case.



Green treasures are safe. The red treasure is lost. The dashed circle is where the lost treasure lands. The black dot is the center.

Problem B. The Rubber Stamp

You have a blank paper strip with n cells, numbered 1 to n from left to right, and a rubber stamp that prints a fixed word P of length k.

Each stamping operation presses the rubber stamp onto the strip, printing the word P across k consecutive cells and overwriting any ink previously there. The stamp may partially hang off either end of the strip; any ink landing outside the strip is lost. Formally, you may place the stamp at any integer position p, so that for each $i \in \{0, 1, \ldots, k-1\}$, if cell $p+i$ is on the strip ($1 \le p+i \le n$), it receives character $P[i]$.

Given the target string T of length n, find the minimum number of stamps needed to make the strip exactly T. If it is impossible to obtain the string T, print -1.

Input

The first line of the input contains an integer t ($1 \le t \le 10^5$), denoting the number of test cases.

Each test case consists of three lines:

- The first line contains two integers n and k ($1 \le k \le n \le 10^6$), denoting the length of the strip and the length of the stamp word, respectively.
- The second line contains the target string T of length n.
- The third line contains the stamp word P of length k.

Both strings T and P consist only of lowercase English letters.

It is guaranteed that the sum of n over all test cases does not exceed 10^6.

Output

For each test case, print a single integer representing the minimum number of stamps needed to make the strip match T, or -1 if it is impossible.

Sample Input	Sample Output
3 5 3 abcba abc 3 3 cba abc 4 3 bbbb abc	3 3 -1

Notes

In the first test case of the sample, one optimal sequence of 3 stamp presses is:

1. Stamp $P =$ `"abc"` at position 3 (covering cells 3..5): the strip becomes `"..abc"`.
2. Stamp $P =$ `"abc"` at position 5 (covering cells 5..7, hanging off the right end so only cell 5 receives $P[0] =$ `'a'`): the strip becomes `"..aba"`.
3. Stamp $P =$ `"abc"` at position 1 (covering cells 1..3): the strip becomes `"abcba"`.

In the second test case of the sample, one optimal sequence of 3 stamp presses is:

1. Stamp $P =$ `"abc"` at position 1 (covering cells 1..3): the strip becomes `"abc"`.
2. Stamp $P =$ `"abc"` at position -1 (covering cells $-1..1$, hanging off the left end so cell 1 receives $P[2] =$ `'c'`): the strip becomes `"cbc"`.
3. Stamp $P =$ `"abc"` at position 3 (covering cells 3..5, hanging off the right end so cell 3 receives $P[0] =$ `'a'`): the strip becomes `"cba"`.

In the third test case of the sample, any stamp placing `'b'` at cell i simultaneously places `'a'` at cell $i-1$ and `'c'` at cell $i+1$, so no two adjacent cells can both be `'b'`. Thus, it is impossible to produce `"bbbb"`, and the answer is -1.

Problem C. In Agreement

You are given an undirected tree with n vertices. Vertex i has a positive integer weight a_i.

You may choose any vertex as the root of the tree and any positive integer k.

After rooting the tree, let s_v denote the sum of the weights of all vertices in the subtree of vertex v, including v itself.

A choice of the root and k is *valid* if all subtree sums have the same remainder modulo k.

Find the maximum possible value of k among all valid choices.

Input

The first line contains a single integer T ($1 \le T \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \le n \le 2 \cdot 10^5$) — the number of vertices.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^6$) — the weights of the vertices.

Each of the next $n-1$ lines contains two integers u and v ($1 \le u, v \le n$, $u \ne v$), denoting an edge between vertices u and v.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the maximum possible value of k.

Sample Input	Sample Output
1 4 1 1 1 1 1 2 1 3 1 4	3

Explanation

Choose vertex 1 as the root and $k = 3$. The subtree sums are $4, 1, 1, 1$, all of which leave remainder 1 modulo 3.

Problem D. Stuck in the Middle

Consider a string t of odd length m. Its middle character is the character at position $\frac{m+1}{2}$ (1-indexed).

We call t *centered* if its middle character appears more than $\frac{m}{2}$ times in t.

You are given a string s consisting of lowercase English letters.

Determine whether every odd-length substring of s is centered.

Input

The first line contains a single integer T ($1 \le T \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 2 \cdot 10^5$) — the length of s.

The second line contains a string s of length n, consisting of lowercase English letters.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print `YES` if every odd-length substring of s is centered. Otherwise, print `NO`.

Sample Input	Sample Output
3 8 centered 5 abbba 5 aabbz	NO YES NO

Explanation

In the first test case, the substring `cen` is not centered, because its middle character `e` appears only once.

In the second test case, every odd-length substring is centered, so the answer is `YES`.

In the third test case, the substring `aabbz` is not centered, because its middle character `b` appears only twice.

Problem E. Perfect Couple

Consider the function

$$f(x, y) = x^3y - xy^3$$

For a positive integer m, two integers x and y are called an m-**perfect couple** if $f(x, y)$ is divisible by m.

You are given an array of n integers $a_1, a_2, \ldots, a_n$ and q queries. Each query consists of three integers l, r and m.
For each query, find an m-perfect couple in the subarray $a_l, a_{l+1}, \ldots, a_r$, or report that there is none. Formally, find two **distinct** indices i and j such that $l \leq i, j \leq r$ and a_i and a_j are an m-perfect couple, or report that no such pair of indices exists.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two space-separated integers n and q ($2 \leq n \leq 10^6$, $1 \leq q \leq 10^5$) — the size of the array and the number of queries.

The second line of each test case contains n space-separated integers $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array.

Each of the next q lines contains three space-separated integers l, r and m ($1 \leq l < r \leq n$, $2 \leq m \leq 100$) — the description of a query.

It is guaranteed that the sum of n over all test cases doesn't exceed 10^6 and the sum of q over all test cases doesn't exceed 10^5.

Output

For each query, output the answer in a line.

If an m-perfect couple exists in the subarray $a_l, a_{l+1}, \ldots, a_r$, output two space-separated integers i and j ($l \leq i, j \leq r$, $i \neq j$) such that a_i and a_j are an m-perfect couple. If there are multiple answers, you may output any of them.

Otherwise, output -1.

Sample Input	Sample Output
2 5 4 16 15 11 9 4 1 3 7 1 4 7 1 3 5 4 5 10 2 2 15 10 1 2 4 1 2 6	-1 1 4 1 2 4 5 -1 1 2

Note

In the first query of the first test case, $f(16, 15) = 7440$, $f(16, 11) = 23760$ and $f(15, 11) = 17160$. None of them is divisible by 7.

In the second query of the first test case, $f(16, 9) = 25200 = 7 \times 3600$. So, a_1 and a_4 are a 7-perfect couple.

In the third query of the first test case, $f(16, 15) = 7440$, $f(16, 11) = 23760$ and $f(15, 11) = 17160$ are all divisible by 5. So, the pairs of indices $(1, 2)$, $(1, 3)$ and $(2, 3)$ are all correct answers.

In the fourth query of the first test case, $f(9, 4) = 2340$, which is divisible by 10.

In the second test case, $f(15, 10) = 18750$, which is divisible by 6, but not by 4.

Problem F. Yet Another Stone Problem

There are N piles of stones, where the i-th pile contains A_i stones. Alice and Bob play a game on these piles. They alternate turns, and Alice moves first. On each turn, the current player must:

1. choose at least 1 and at most K non-empty piles, and
2. remove a positive number of stones from each chosen pile.

The number of stones removed may differ between the chosen piles. The player who cannot make a move loses. Both players play optimally.

You are given N, K, S and the name of a player *target*. Construct positive integers $A_1, A_2, \ldots, A_N$ such that

- $A_1 + A_2 + \cdots + A_N = S$, and
- if the piles initially contain $A_1, A_2, \ldots, A_N$ stones, the player named *target* wins the game.

If no such sequence exists, report it.

Input

The first line of the input contains an integer T ($1 \le T \le 300$) denoting the number of test cases. Each of the next T lines contains three integers N, K, S ($1 \le N \le 10^5$, $1 \le K \le 10^9$, $1 \le S \le 10^{18}$) and a string *target*, which is either `Alice` or `Bob`.

The sum of N over all test cases does not exceed $5 \cdot 10^5$.

Output

For each test case, print a single line. If no valid sequence exists, print -1. Otherwise, print N positive integers $A_1, A_2, \ldots, A_N$ separated by spaces. If there are multiple valid answers, you may print any of them.

Sample Input	Sample Output
3 1 1 2 Alice 1 1 2 Bob 2 1 2 Bob	2 -1 1 1

Explanation

In the first test case, the only possible position is a single pile of 2 stones. Alice removes both stones in her first move, and Bob cannot move, so Alice wins.

In the second test case, the only possible position is again a single pile of 2 stones, where Alice wins. So there is no position where Bob wins.

In the third test case, the only possible position is two piles of 1 stone each. Since $K = 1$, Alice must take one of the piles, Bob takes the other one, and Alice cannot move. So Bob wins.

Problem G. Optimal Deployment

A software platform contains N modules connected by $N-1$ bidirectional API links. The links form a connected graph with no cycles (a tree).

Choose any module as the root (the entry point), which determines a parent-child relationship for every link.

For a node u in the rooted tree, let subtreeSize(u) be the number of modules in the subtree rooted at u, including u itself. Let childCount(u) be the number of direct children of u, not including u.

There are M types of deployment servers. Server type i has capacity C_i and costs P_i per deployment operation. A server type can be reused any number of times, with its cost charged for each operation.

To deploy a module u, choose one of the following operations:

- **Deploy the entire subtree of u.** One operation deploys u and every descendant of u. The chosen server must have capacity $C_i \geq$ `subtreeSize`(u). No further operations are needed for any module in this subtree.
- **Deploy only u.** One operation deploys u and no other module. The chosen server must have capacity $C_i \geq$ `childCount`(u) (the number of u's direct children). Afterward, each direct child of u must be deployed separately, using either operation.

Find the minimum total cost to deploy every module, choosing both the entry point and the server type for each operation optimally. If deployment is impossible under every choice of entry point and available server types, output -1.

Input

The first line contains T ($1 \leq T \leq 25$), the number of test cases.

For each test case, the first line contains M and N ($1 \leq M, N \leq 10^5$), the number of server types and modules, respectively.

The second line contains the M server capacities $C_1, C_2, \ldots, C_M$ ($1 \leq C_i \leq 10^9$). The third line contains their corresponding costs $P_1, P_2, \ldots, P_M$ ($1 \leq P_i \leq 10^9$).

Each of the next $N-1$ lines contains two integers u and v ($1 \leq u, v \leq N$, $u \neq v$), describing a bidirectional API link between modules u and v.

The sum of M over all test cases does not exceed 5×10^5, and the sum of N does not exceed 5×10^5.

Output

For each test case, output one integer: the minimum total deployment cost, or -1 if deployment is impossible.

Sample Input	Sample Output
1 4 5 1 2 3 4 4 9 11 20 1 2 2 3 2 4 2 5	24

Explanation

Choose module 1 as the entry point.

Module 1 has one direct child, so deploy only module 1 using a server with capacity 1 and cost 4.

The subtree rooted at module 2 contains modules $2, 3, 4$, and 5. Since subtreeSize(2) is 4, deploy this entire subtree in one operation using a server with capacity 4 and cost 20.

The total cost is therefore $4 + 20 = 24$.

Problem H. Vanishing Fairy Tale

You are given an array of N integers $A_1, A_2, \ldots, A_N$.

A **spell** works as follows. First, choose a subsequence of the array (possibly empty). Then, for **each** chosen element A_i, choose exactly one of three fates:

- keep it as $+A_i$,
- flip its sign, turning it into $-A_i$, or
- make it vanish, turning it into 0.

The result of the spell is the sequence of the transformed chosen elements. Its **beauty** is the number of distinct integers in it. Note that every vanished element becomes the same integer 0, so all vanished elements together add at most one distinct integer. The beauty of an empty result is 0.

Two spells are considered different if some index i is chosen in one of them but not in the other, or is chosen in both but gets a different fate. In particular, there are exactly 4^N different spells.

You have to answer Q queries. In each query you are given an integer K. Count the spells whose beauty is a multiple of K. Note that 0 is a multiple of every K.

Input

The first line contains a single integer T ($1 \le T \le 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains two integers N and Q ($1 \le N, Q \le 10^5$) — the size of the array and the number of queries.

The second line of each test case contains N integers $A_1, A_2, \ldots, A_N$ ($1 \le A_i \le N$).

Each of the next Q lines of each test case contains a single integer K ($1 \le K \le 10^5$).

It is guaranteed that the sum of N over all test cases does not exceed $2 \cdot 10^5$, and the sum of Q over all test cases does not exceed $2 \cdot 10^5$.

Output

For each query of each test case, print the number of spells whose beauty is a multiple of K, modulo 998244353, on its own line.

Sample Input	Sample Output
1 3 3 2 1 2 1 2 3	64 35 15

Note

In the sample, there are $4^3 = 64$ spells. Their beauties are distributed as follows:

- 1 spell has beauty 0 (nothing is chosen);
- 15 spells have beauty 1;

- 34 spells have beauty 2;
- 14 spells have beauty 3.

For example, choosing all three elements and giving them the fates "vanish", "keep", "vanish" produces the sequence $0, 1, 0$, which has beauty 2.

For $K = 1$, all 64 spells are counted. For $K = 2$, the spells with beauty 0 or 2 are counted: $1 + 34 = 35$. For $K = 3$, the spells with beauty 0 or 3 are counted: $1 + 14 = 15$.